OSTRAVSKÁ UNIVERZITA
ÚSTAV PRO VÝZKUM A APLIKACE FUZZY MODELOVÁNÍ

7LLMA · TAHÁK PRO STUDENTY

Lokální inference: enginy a API

LM Studio · Ollama · vLLM · llama.cpp · Open WebUI · FastAPI

k decku 7LLMA-4-lokalni-inference-v2.0 · stav 22. 9. 2026

Mám model… a?

Vstup → tokenizér → model (váhy) → **serving engine** → API → aplikace. Model je jen soubor; engine ho načte do (V)RAM a spouští dopředný průchod pro každý token. Serving řeší rychlost, spolehlivost a cenu za token.

- **Kdo dělá co:** desktop = klikat sám (LM Studio, Jan, AnythingLLM) · engine = tělo (Ollama, llama.cpp, vLLM, SGLang) · UI = vychytávky pro lidi (Open WebUI, LibreChat) · vlastní API = vaše pravidla (FastAPI, LiteLLM proxy).
- **Stejné rozhraní všude:** OpenAI-kompatibilní HTTP API – co funguje lokálně, funguje na pronajatém GPU i v cloudu.

Proč lokálně, kdy ne

- **Proč:** vůbec spustit otevřené váhy (Gemma, Qwen, GLM…) · data neopouštějí stroj (GDPR, firemní tajemství) · cena při tisících dotazů denně · kontrola (verze modelu se nemění pod rukama) · integrace do IDE, RAG, edge.
- **Kdy ne:** frontier kvalita, žádný hardware, jeden dotaz týdně – API je levnější a lepší.

Pojmy

Pojem	Co to je
Prefill / decode	prompt najednou (paralelně, rychle) / token po tokenu (pomalu)
TTFT / tok/s	čas do prvního tokenu / tokeny za sekundu; měřit zvlášť
KV cache	paměť attention přes kontext; roste s délkou × souběhem; často víc než váhy
Batching	víc požadavků v jednom průchodu; statický vs. continuous
Kvantizace	váhy v méně bitech (FP16 → Q8 → Q4): menší, rychlejší, mírně horší
Offloading	část vrstev na GPU, zbytek v RAM (`-ngl`); funguje, 5–10× pomaleji

Orientace: 8B Q4 na RTX 4070 ≈ 60 tok/s pro jednoho; CPU notebook 5–10 tok/s; čitelná rychlost ≈ 15 tok/s.

Soubory modelů

Formát	Co	Kdo čte
safetensors	váhy + metadata, bezpečný, sharded	HF transformers, vLLM, SGLang
GGUF	váhy + tokenizér + metadata v jednom; kvantizovaný	llama.cpp, Ollama, LM Studio
Modelfile	recept: FROM, PARAMETER, SYSTEM, TEMPLATE	Ollama
AWQ / GPTQ / FP8	kvantizované váhy pro GPU; na CPU ne	vLLM, TensorRT-LLM

- **GGUF** pro CPU, notebook, Apple Silicon, edge; **safetensors (+ FP8/AWQ)** pro GPU server. Konverze `convert_hf_to_gguf.py`; kvantizaci nevrátíte.
- HF repo = config, tokenizer, `*.safetensors`, model card (licence, prompt formát). Tokenizér cestuje s modelem: GGUF ho má uvnitř, u HF repa stáhnout spolu s váhami.

Hardware a backendy

Hardware	Backend	Co zvládne
NVIDIA GPU	CUDA	vše; vLLM ≥ 8 GB, ideálně 24+
AMD GPU	ROCm, Vulkan	llama.cpp, Ollama, vLLM omezeně
Apple Silicon	Metal, MLX	sdílená paměť 32–128 GB = velké modely
Intel, iGPU	Vulkan, SYCL	llama.cpp, Ollama (`OLLAMA_VULKAN=1`)
Jen CPU	AVX2/512	4–8B Q4 čitelně, 30B pomalu

Paměť = parametry × bajty (Q4 ≈ 0,55 B, FP16 = 2 B) + KV cache pro kontext + rezerva. Rozhoduje paměť, ne výkon.

Pronajaté GPU (RunPod, Vast.ai, Lambda): stejné nástroje, jen `ssh` navíc.

LM Studio

Desktop pro Windows, macOS, Linux, zdarma i pro firmy. Discover = katalog z Hugging Face (GGUF, MLX) s „fits in GPU“; chat s presety, soubory, obrázky, MCP; pod kapotou llama.cpp nebo MLX. **Kdy:** jeden člověk, notebook, klikat a experimentovat.

```
lms server start --port 1234    # nebo GUI
lms get google/gemma-4-E4B-it-GGUF
lms load gemma-4-e4b --context-length 8192
lms daemon up           # headless, bez GUI
# API: :1234/v1/chat/completions, /v1/messages
```

Výchozí kontext 4K – nastavit při načtení. Stejný `curl` na `/v1/chat/completions` funguje na LM Studio (1234), Ollama (11434), vLLM (8000) i llama-server (8080).

Ollama

Open source (MIT), nad ggml/llama.cpp; model = tag jako u Dockeru (`gemma4:e4b`, `qwen3:8b-q4_K_M`); registry ollama.com/library + `hf.co/…`. Server na 11434: nativní `/api/chat`, `/api/generate`, `/api/embed`, `/api/tags`, `/api/ps` + OpenAI `/v1/chat/completions` + Anthropic `/v1/messages`. **Sweet spot:** 1–5 uživatelů, jeden stroj, nejvíc integrací.

```
ollama pull gemma4:e4b      # stáhnout
ollama run gemma4:e4b       # chat; "…" jednorázově
ollama ls / ps / show / stop / rm
ollama create muj -f Modelfile
ollama launch               # Claude Code, Codex…
ollama run hf.co/bartowski/Qwen3-8B-GGUF:Q4_K_M
# v chatu: /set parameter num_ctx 16384
OLLAMA_HOST=0.0.0.0:11434  # naslouchat v síti
OLLAMA_KEEP_ALIVE=30m      # držet v paměti
OLLAMA_NUM_PARALLEL=4      # souběžné požadavky
```

```
# přenositelný klient: mění se jen base_url
from openai import OpenAI
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama")          # libovolný string
r = client.chat.completions.create(
    model="gemma4:e4b", temperature=0,
    messages=[{"role": "user", "content": "Ahoj"}])
print(r.choices[0].message.content)
```

Nativní knihovna: `ollama.chat(model, messages, options={"num_ctx": 8192}, format=schema, stream=True)`, `ollama.embed(...)`.

```
FROM gemma4:e4b          # nebo ./vahy-Q4_K_M.gguf
PARAMETER temperature 0.2
PARAMETER num_ctx 16384
SYSTEM """Jsi asistent podpory. Když nevíš: NEVÍM."""
```

Modelfile: `ollama create podpora -f Modelfile`, `ollama push ucet/podpora`. Vlastní fine-tune: `convert_hf_to_gguf.py` → `FROM ./muj.gguf` + `TEMPLATE`.

POZOR Ollama pasti: kontext 4K – delší prompt se tiše uřízne (`num_ctx`, `OLLAMA_CONTEXT_LENGTH`) · model se po 5 min vyloží (`keep_alive: -1`) · `OLLAMA_HOST=0.0.0.0` bez autentizace = kdokoli v síti maže modely · deset uživatelů = fronta (`NUM_PARALLEL` do ~5, pak vLLM) · `:cloud` modely nejsou lokální · tag bez kvantizace = Q4_K_M, pro kvalitu `q8_0`.

vLLM

Open source (Apache 2.0), Python + CUDA. **PagedAttention** (KV cache po stránkách, bez fragmentace → 2–4× víc souběžných sekvencí) a **continuous batching** (nový požadavek vstupuje ihned, GPU nečeká). V1: prefix caching, chunked prefill, speculative decoding, tensor parallel, FP8/AWQ/GPTQ. safetensors z HF, GGUF jen experimentálně; NVIDIA především.

```
uv pip install vllm     # Linux, NVIDIA, Py 3.10–3.12
vllm serve Qwen/Qwen3-8B --max-model-len 16384 \
  --gpu-memory-utilization 0.9 --port 8000
vllm serve Qwen/Qwen3-8B-AWQ --quantization awq
docker run --gpus all -p 8000:8000 \
  -v ~/.cache/huggingface:/root/.cache/huggingface \
  vllm/vllm-openai:latest --model Qwen/Qwen3-8B
curl localhost:8000/v1/models   # metriky: /metrics
```

- **1 uživatel ≈ llama.cpp; 50 uživatelů ≈ 10× propustnost.** Klient stejný (OpenAI SDK, `base_url`); 32 souběžných vláken server zpracuje v jedné dávce. Offline bez HTTP: `LLM(model).generate(prompts)`.
- **Kdy:** tým, API pro aplikaci, noční dávky; NVIDIA 24 GB+ (8B FP16), 80 GB (70B FP8); produkce s metrikami a Kubernetes. **Kdy ne:** notebook, jeden uživatel, CPU, GGUF, sdílené GPU (zabere 90 % VRAM); start desítky sekund. Stejná třída: SGLang, TensorRT-LLM, TGI.

llama.cpp

Open source (MIT), C/C++ bez závislostí, knihovna ggml; GGUF, kvantizace Q2–Q8, K-quants, IQ; backendy CPU, CUDA, Metal, Vulkan, ROCm, SYCL; Raspberry Pi i H100. Pod kapotou Ollamy, LM Studia, Jan; ~2× rychlejší než Ollama pro jednoho.

```
cmake -B build -DGGML_CUDA=ON && cmake --build build
./build/bin/llama-cli \
  -hf ggml-org/gemma-4-E4B-it-GGUF:Q4_K_M \
  -ngl 99 -c 8192 -p "Vysvětli KV cache."
./build/bin/llama-server -hf … -ngl 99 -c 8192 \
  --host 0.0.0.0 --port 8080 --api-key tajny
# web UI :8080, API /v1/chat/completions
./build/bin/llama-bench -m model.gguf -ngl 99
```

- `-ngl 99` = všechny vrstvy na GPU; `-c` = kontext; `-hf` stáhne GGUF z Hugging Face; `llama-bench` dá tok/s pro prefill (pp) a decode (tg). Binárky: GitHub Releases, brew, winget, Docker `ghcr.io/ggml-org/llama.cpp:server`.
- **Router mode:** `llama-server --models-dir ~/models --models-max 2 --models-autoload` bez `-m` → jeden port, víc modelů, klient volí polem `model`, LRU. **llama-swap:** externí proxy (`config.yaml`: `models → cmd, ttl`, `groups`), přepíná llama-server i vLLM; OpenAI + Anthropic.

Kvantizace GGUF – co vybrat

Typ (bitů)	8B	Kvalita	Kdy
F16 (16)	16 GB	referenční	máte VRAM; měření kvality
Q8_0 (8)	8,5 GB	≈ F16	když se vejde; pro kvalitu
Q6_K (6,5)	6,6 GB	téměř Q8	dobrý kompromis
Q4_K_M (4,8)	4,9 GB	mírná ztráta	výchozí volba všude
Q3_K_M / IQ3	3,9 GB	znatelná	jen když se Q4 nevejde
Q2_K / IQ2	3,0 GB	velká	experimenty; malé modely ne

Větší model v Q4 je lepší než menší v F16 při stejné paměti (30B Q4 > 8B F16). Kvantizace bolí nejvíc pod 7B a u reasoning modelů. Vlastní: `llama-quantize in.gguf out.gguf Q4_K_M`; imatrix zlepší Q3/Q2. Vždy změřit na svých 30 dotazech.

Srovnání čtyř nástrojů

	Ollama	llama.cpp	vLLM	LM Studio
Cíl	jedno kliknutí, server	low-level, edge, výkon pro 1	propustnost, produkce	klikání, desktop
Uživatelů	1–5	1 (router: pár)	desítky–stovky	1
Hardware	CPU / GPU / Apple	cokoli vč. Raspberry Pi	NVIDIA GPU (AMD)	CPU / GPU / Apple
Formát	GGUF (+ HF import)	GGUF	safetensors, FP8/AWQ	GGUF, MLX
API	nativní + OpenAI + Anthropic	OpenAI (+ Anthropic)	OpenAI	OpenAI + Anthropic
1 / 50 uživatelů	rychlé / fronta	nejrychlejší / fronta	rychlé / 10× propustnost	rychlé / ne
Start	sekundy	sekundy	desítky sekund	sekundy
Kdy	prototyp, výzkum, domácí server	edge, CPU, sdílené GPU, max výkon	API pro aplikaci, dávky, tým	experimenty, jeden člověk

Rozhodovací strom

Kolik · hardware · styl	Nástroj
1 · notebook · klikat	LM Studio
1 · notebook · skript / API	Ollama (nebo llama-server)
1 · CPU / edge / max výkon	llama.cpp
1–5 · domácí server · integrace	Ollama + Open WebUI
10+ · GPU server · aplikace	vLLM (SGLang)
více modelů na jednom stroji	llama-server router / llama-swap

Nejčastější správná kombinace: **Ollama na notebooku pro vývoj, vLLM na serveru pro nasazení** – stejný klientský kód.

Další: SGLang (RadixAttention) · TensorRT-LLM, Triton (max výkon NVIDIA) · TGI, LMDeploy · **LiteLLM** proxy (jeden OpenAI formát nad 100+ API; klíče, limity, logy) · MLC LLM / WebLLM (telefon, prohlížeč) · Unsloth, Axolotl = fine-tuning, ne serving.

Docker a Open WebUI

```
services:
  ollama:
    image: ollama/ollama
    ports: ["11434:11434"]
    volumes: ["ollama:/root/.ollama"]
    # GPU: deploy.resources.reservations.devices
  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    ports: ["3000:8080"]
    environment:
      OLLAMA_BASE_URL: http://ollama:11434
```

- NVIDIA v Dockeru: `nvidia-container-toolkit`, `--gpus all`; ověřit `nvidia-smi` uvnitř kontejneru. Proč: pinované verze CUDA a enginu, stejný compose na notebooku i serveru.
- **Open WebUI:** nejrozšířenější self-hosted chat UI; passthrough na Ollama, vLLM, llama-server, LM Studio i cloud; uživatelé, role, SSO; RAG nad dokumenty; nástroje, MCP; Pipelines v Pythonu. Port 3000, první účet = admin; Connections přidá vLLM (`http://gpu:8000/v1`). **První den:** výchozí model a systémový prompt · registrace jen pro pozvané · vícejazyčný embedding (bge-m3) · HTTPS přes reverse proxy.

Lokální model v IDE

```
ollama pull qwen3-coder:30b
export ANTHROPIC_BASE_URL=http://localhost:11434
export ANTHROPIC_AUTH_TOKEN=ollama
export ANTHROPIC_MODEL=qwen3-coder:30b
claude          # nebo: ollama launch claude
```

VS Code: Copilot BYOK, Continue (`provider: ollama`), Cline / Roo Code (`localhost:11434/v1`). 30B coder stačí na malé úlohy, frontier kvalita to není.

Vlastní API: FastAPI + Uvicorn

Proč vrstva nad enginem: autentizace a limity · prompty na jednom místě · validace vstupu i výstupu (Pydantic) · logování a metriky · výměna modelu / enginu bez změny klientů. Vzor: klient → FastAPI (pravidla) → engine → model; `/docs` = dokumentace zdarma.

```
class SumIn(BaseModel):
    text: str = Field(min_length=20,
                      max_length=20_000)
    vet: int = Field(default=3, ge=1, le=10)

@app.post("/summarize", response_model=SumOut)
def summarize(inp: SumIn):
    prompt = (f"Shrň text v <text> do {inp.vet} "
              f"vět.\n<text>{inp.text}</text>")
    try:
        r = ollama.chat(model=MODEL,
            options={"temperature": 0},
            messages=[{"role": "user",
                       "content": prompt}])
    except Exception as e:
        raise HTTPException(502, f"engine: {e}")
    return SumOut(shrnuti=r.message.content,
                  model=MODEL)    # uvicorn main:app
```

- Pydantic hlídá vstup dřív, než se zaplatí tokeny; 502 místo 500 (chyba enginu není chyba API); systémový prompt patří sem, ne do klienta.
- **Streaming a výměna enginu:** `AsyncOpenAI(base_url=os.environ["ENGINE_URL"])` + `StreamingResponse(gen())`, kde `gen()` dává `chunk.choices[0].delta.content`. Ollama na notebooku, vLLM na serveru – jeden kód, jiná proměnná prostředí.
- **Dál:** pytest s mockem enginu a regresní sadou · structured output se schématem · API klíč (`Depends`), rate limit, `max_tokens`, timeout, log `usage` · LiteLLM místo vlastního routeru.

Provoz, bezpečnost, pasti

- **Měřit tok/s a TTFT** (`llama-bench`, vLLM `/metrics`, usage v API) před a po každé změně; souběh: `NUM_PARALLEL`, `max-num-seqs`.
- **Engine nikdy přímo do sítě:** `--api-key` (vLLM, llama-server), reverse proxy s TLS. **Modely jsou kód:** jen ověřené zdroje (ollama.com, ggml-org, unsloth, bartowski); pickle `.bin` nikdy. Prompt injection přes RAG a nástroje platí i lokálně.
- Jeden model stále v paměti > dva, které se střídají. Enginy vydávají týdně: pinovat verze, testovat před upgradem.

POZOR Pasti: kontext 4K jako výchozí · model se nevejde → tichý offloading, 10× pomaleji (`ollama ps`: „48 %/52 %“) · vlastní GGUF bez chat template = nesmysly · reasoning model bez `think: false` přemýšlí dlouho a draze · Q4 vs. Q8 = jiné odpovědi, testovací sadu po každé změně · dva enginy na kartě = OOM · Docker bez `--gpus all` běží na CPU.

TIP Na zapamatování: model je soubor; engine ho rozhýbe; UI přidá lidi; vlastní API přidá pravidla. Rozhoduje paměť. LM Studio = klikat sám · Ollama = server pro 1–5 lidí · llama.cpp = edge, CPU, max výkon pro jednoho · vLLM = propustnost pro desítky na GPU. Všechno mluví OpenAI API: klient je stejný na notebooku, serveru i v cloudu.

Zdroj: přednáška Lokální inference (7LLMA-4-lokalni-inference-v2.0.pptx), stav k 22. 9. 2026.